

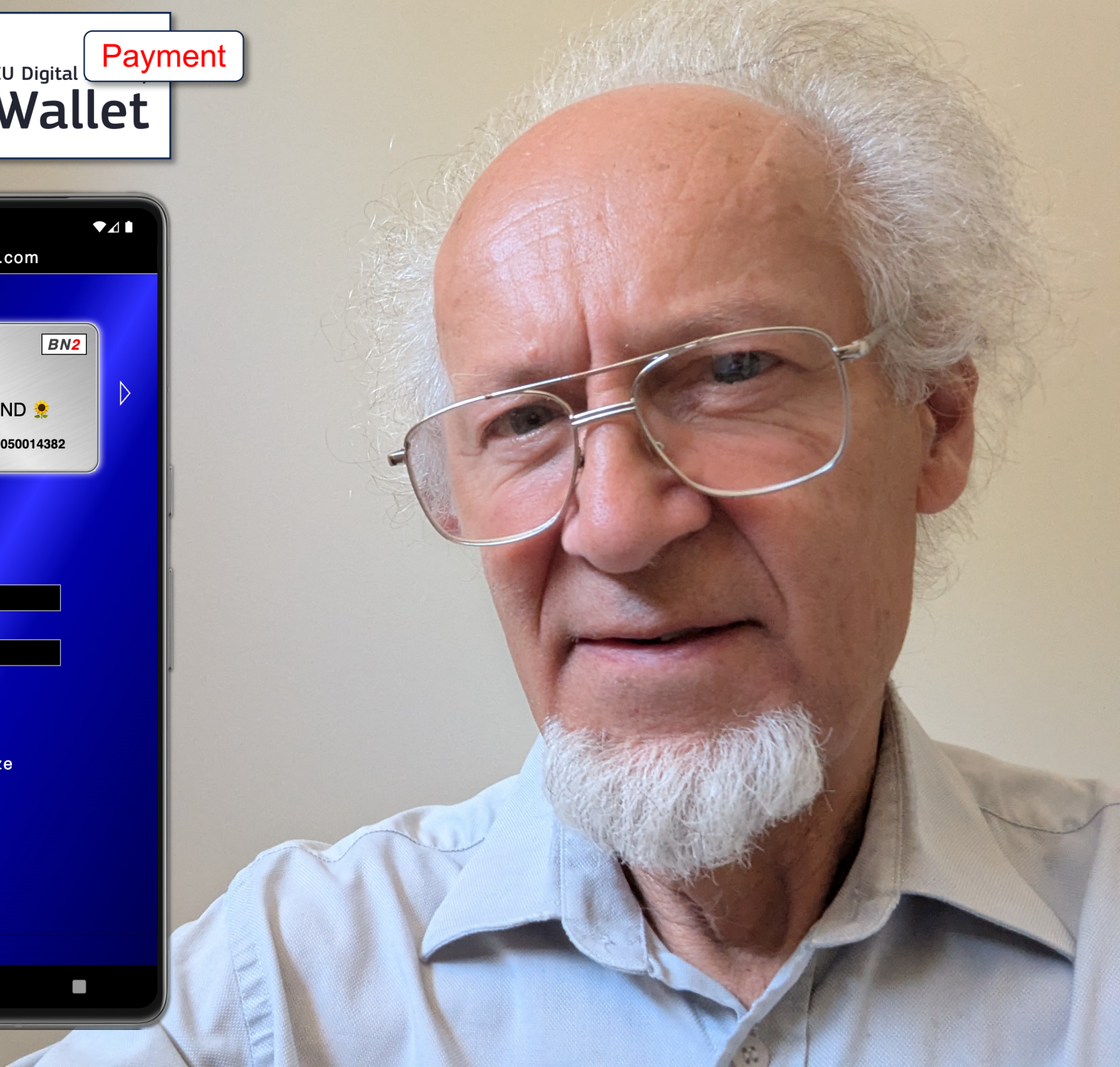
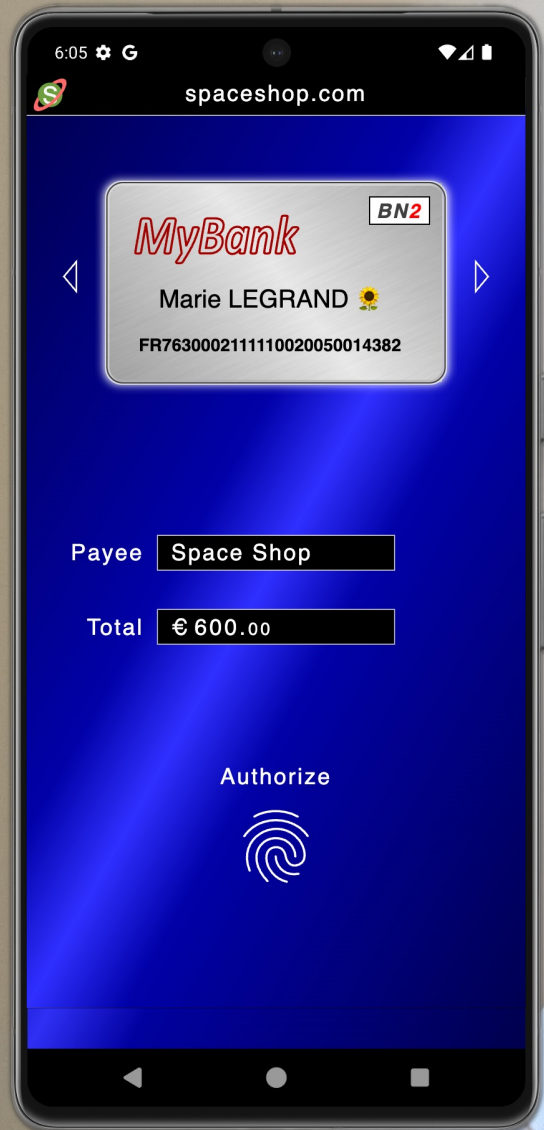
PAYMENT

EU Digital ~~Identity~~ Wallet

Implementation Suggestions

Payments << >> Identity

Anders Rundgren, September 2026



Standardization Activities

RFC 8785 – *Principal designer*

JSON Canonicalization Scheme (JCS)

CBOR::Core – *Principal designer*

<https://datatracker.ietf.org/doc/draft-rundgren-cbor-core/>

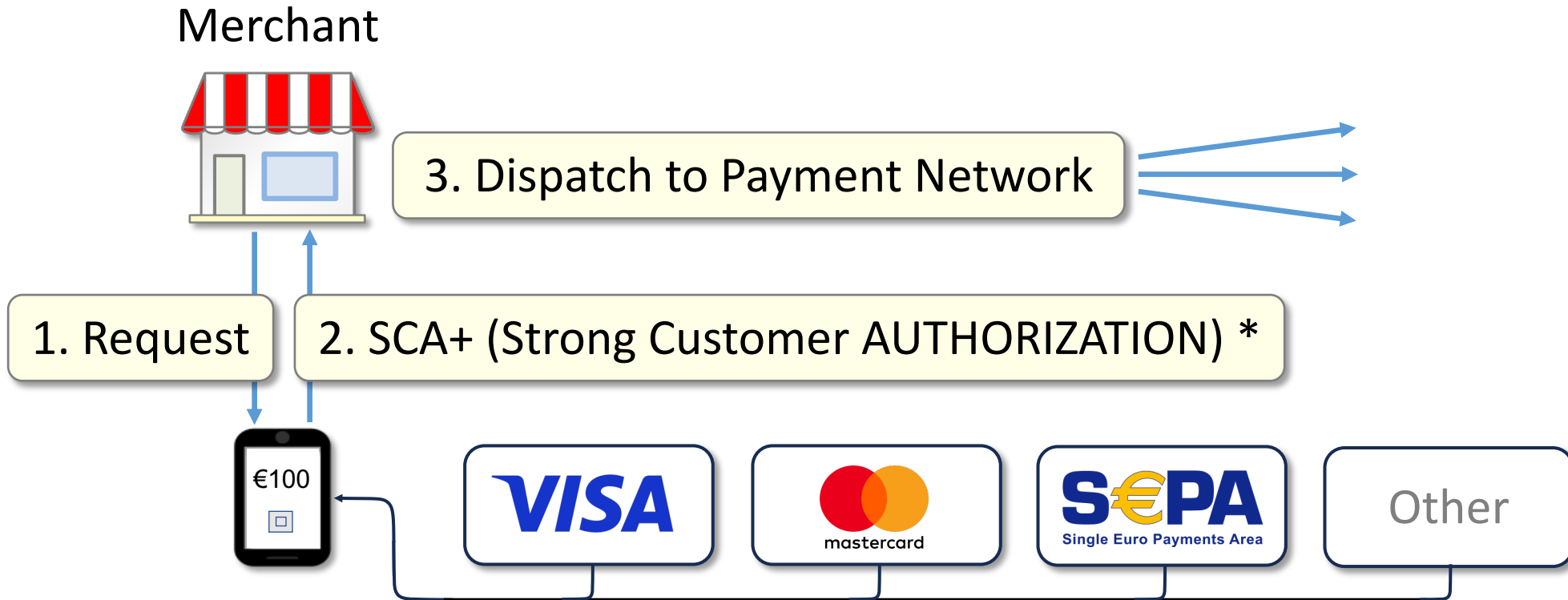
Cross-platform, Deterministically Encoded CBOR (“JSON Challenger”)

Implementations: Java, Android, JavaScript, Python, and Rust

Enabling: Embedded Signatures (“COSE Challenger”)

EMV® “On Steroids”

This invention is hereby placed in the public domain



- * *Payer-signed* {
- Payment request data in *clear*
 - Routing/Network data in *clear*
 - *Issuer-encrypted* PII-related authorization data

e-Receipts – Invention Disclosure

1. When a *User* clicks a “Pay” button or swipes their phone, the *Merchant* activates the *User*’s payment *Wallet* with a *payment request* message including a *randomized receiptUrl*.

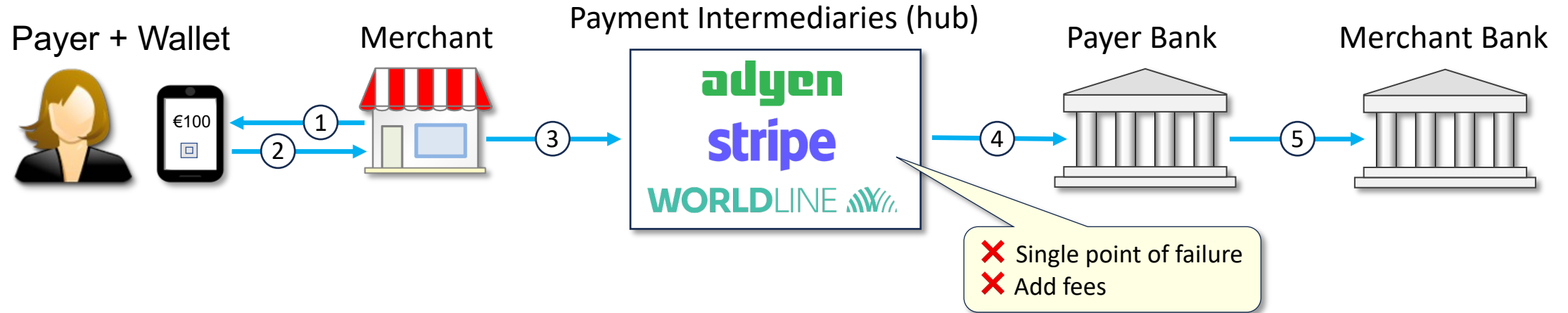
```
{
  "paymentRequest": {
    "commonName": "Space Shop",
    "amount": "550.00",
    "currency": "EUR",
    "referenceId": "20231007"
  },
  "receiptUrl": "https://spaceshop.com/receipts/20231007j5lOEL2w9cWBFUwkbrFgjQ"
}
```

Non-normative sample request

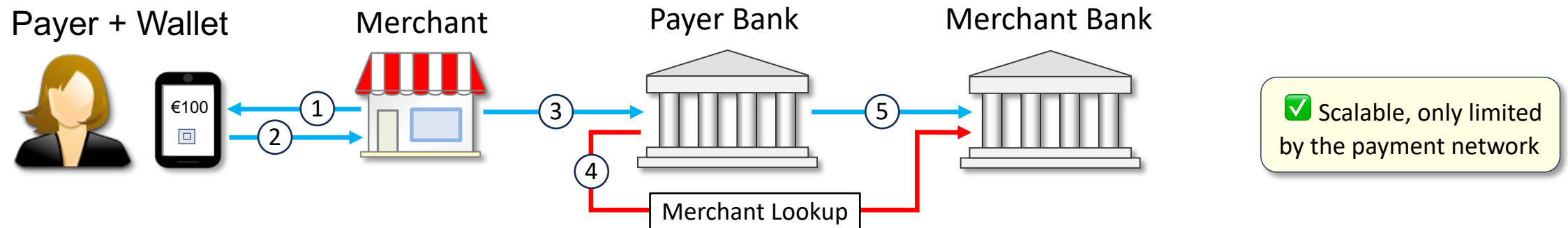
2. After authorization by the *User*, the *Wallet* responds with the authorization data.
3. The Payment process. *Details are outside of this specification.*
4. If step #3 is successful, the *Merchant* provides a *Receipt* at the *receiptUrl*. Note that a *Receipt* may have states like **ready** and **inProgress** since gas station payments are not finalized until the pump operation stops.
5. If step #3 is successful, the *Merchant* sends a confirmation message to the *Wallet*.
6. When the confirmation has been received, the *Wallet* *polls* the *receiptUrl* for the *Receipt* which is subsequently stored in the *Wallet* for easy retrieval by the *User*.



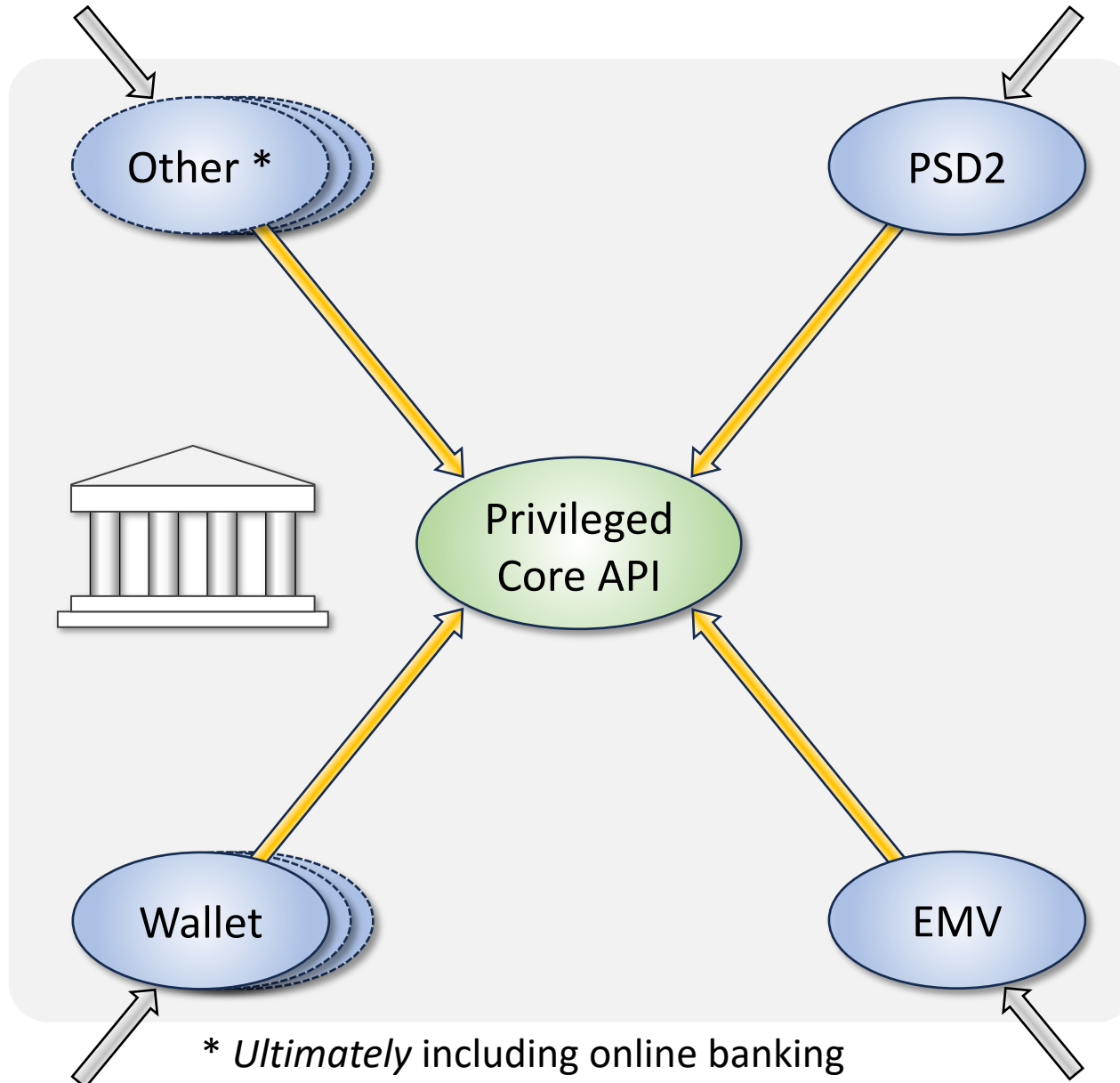
Classic A2A



Resilient & Scalable A2A



Open Banking 2.0 Architecture



Account or payment application using a networked (*loosely coupled*) interface to the core API. This arrangement enables *third-parties* supplying such applications as *boxed solutions, containers, or virtual machines*. Due to the *standardized* core API and security solution, the very same solution should work in *all banks* supporting the Open Banking 2.0 concept. Apart from *reducing cost*, the goal is also about providing a robust foundation for *innovation* and *swift adoption* of new APIs.

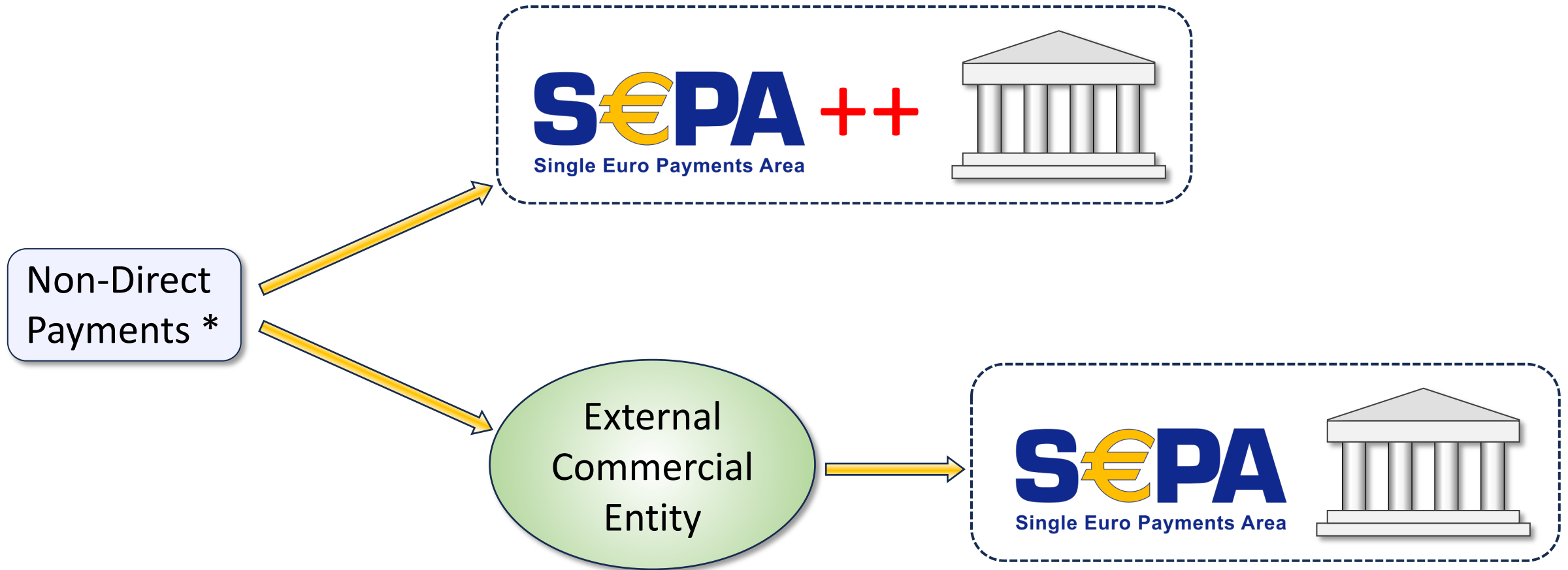
Privileged core (low-level) banking API:

- Enumerate accounts
- Get account information
- Move money (transact)
- Reserve money
- Resolve reservation (transact)
- Identity/authentication services
- ...

→ Internal-only core API and security solution

→ Application-specific API and security solution(s)

Next Step for A2A Payment Systems



* Gas-station payments, subscriptions, deposits, etc.